السلام عليكم ورحمة الله وبركاته.. وبعد،

بين يديك ملخص شامل ومنظم لمادة CPCS-331 _ Chapter 2-13، تم إعداده بعناية ليكون عوناً لك في فهم المادة .وقبل البدء، أضع بين يديك ميثاقاً للأمانة العلمية والتعاون:

⚠️ ميثاق الأمانة والمسؤولية:

هذا العمل هو جهد طالب، أضعه بين يديك بنية النفع والفائدة؛ لذا:

- لا أسامح ولا أحلل استخدامه في الغش أو أي عمل ينافي الأخلاق الإسلامية.
- يُمنع منعاً باتاً تداوله كمصدر تجاري للباعة أو استخدامه في أدوات الذكاء الاصطناعي لغرض التحايل او الغش.
- الأمانة العلمية تقتضي منك استخدامه كمرجع للفهم والمراجعة، لا كبديل عن الضمير والاجتهاد الشخصي.

📣 التواصل معي:

الكمال لله وحده، لذا يسعدني استقبال ملاحظاتكم، تعديلاتكم، أو إضافاتكم عبر:

- الرقم المباشر: 0530717788
- الموقع الشامل: kau-one.web.app

توجيه إيماني:

قبل أن تبدأ، استعذ بالله من الشيطان الرجيم، وجدد نيتك بأن يكون هذا الوقت في سبيل طلب العلم النافع الذي يرفع قدرك في الدنيا والآخرة. قال رسول الله ﷺ: "مَنْ سَلَكَ طَرِيقًا يَلْتَمِسُ فِيهِ عِلْمًا سَهَّلَ اللَّهُ لَهُ بِهِ طَرِيقًا إِلَى الْجَنَّةِ" (رواه مسلم).

إعداد الطالب: إلياس حسان بابور الدفعة: 24 — تم اخذ الماده و عمل الملخص على السلايدات سنه م2026

1. Definitions of AI and Intelligence

- **Artificial Intelligence (John McCarthy):** The science and engineering of making intelligent machines, especially intelligent computer programs.
  - **الذكاء الاصطناعي:** علم وهندسة صنع آلات ذكية، وخاصة برامج الكمبيوتر الذكية.
- **Intelligence:** The computational part of the (human) ability to achieve goals in the world. Varying degrees occur in humans, animals, and some machines.
  - **الذكاء:** الجزء الحسابي من القدرة (البشرية) على تحقيق الأهداف في العالم.
- **Intelligent Behavior Involves:**
  - Ability to act in complex environments (التصرف في بيئات معقدة).
  - Ability to learn from experience (التعلم من التجربة).
  - Ability to think and reason (التفكير والاستنتاج).
  - Ability to perceive relations (إدراك العلاقات).
  - Ability to use tools (استخدام الأدوات).

2. Rational Decisions (Computational Rationality)

A more accurate title for this course would be **Computational Rationality**—which centers on maximizing expected utility.

- **Rational:** Maximally achieving pre-defined goals.
  - **عقلاني:** تحقيق الأهداف المحددة مسبقًا بأقصى حد ممكن.
- **Decisions over Thoughts:** Rationality strictly concerns *what* decisions are made, not the underlying thought process behind them.
  - العقلانية تهتم فقط بالقرارات المتخذة، وليس بعملية التفكير وراءها.
- **Utility:** Goals are expressed in terms of the utility of outcomes. Being rational means maximizing your expected utility.
  - **المنفعة:** التعبير عن الأهداف بدلالة منفعة النتائج، والعقلانية تعني تعظيم هذه المنفعة المتوقعة.

3. The Four Main AI Approaches

The ultimate objective across all these approaches is to build autonomous intelligent machines.

Category (الفئة)	Focus (التركيز)	Approach Name & Description (المنهج والوصف)
Think like people (التفكير كالبشر)	**Thinking Humanly**	Cognitive Science Approach: Simulates human-like thinking and theories of the mind. Focuses on how the program operates internally, not just the outcome (e.g., Chess programs succeed but don't explain how humans actually play chess). نهج العلوم المعرفية: محاكاة التفكير البشري، والتركيز على "كيفية" عمل العقل (العملية) وليس النتيجة فقط.
Act like people (التصرف كالبشر)	**Acting Humanly**	Turing Test Approach: Builds machines to perform tasks humans currently do better. Focuses on what results are achieved rather than intelligent behavior behind the scenes. Used heavily for evaluating Natural Language Generation (NLG) systems. نهج اختبار تورنغ: التركيز على الأفعال والنتائج المشابهة للبشر دون الاهتمام بكيفية تحقيقها.
Think rationally (التفكير بعقلانية)	**Thinking Rationally**	Laws of Thought Approach: Uses computational models and formal logic to understand the mind. Focuses on provably correct inference mechanisms that guarantee an optimal solution. نهج قوانين الفكر: استخدام النماذج الحسابية والقواعد المنطقية لضمان استنتاجات صحيحة وحلول مثالية.
Act rationally (التصرف بعقلانية)	**Acting Rationally**	Rational Agent Approach: Doing the right things to maximize goal achievement. Emulates intelligent behavior sufficiently. Imperfect reasoning is acceptable as long as the primary goal is met. نهج الوكيل العقلاني: التركيز على تحقيق الأهداف بكفاءة (النتائج)، ويُقبل التفكير غير المثالي إذا تحقق الهدف المنشود.

4. History of AI

– Boolean Circuit of the Brain - 1943
McCulloch & Pitts proposed the first mathematical model of neural networks.

(نموذج الدوائر المنطقية للدماغ من قبل مكولوتش وبيتس)

– Computing Machinery and Intelligence - 1950
Alan Turing published his foundational paper introducing the Turing Test.

– Early AI Programs - 1950s
Creation of Samuel's checkers program and Newell & Simon's Logic Theorist.

– Dartmouth Meeting - 1956
The term "Artificial Intelligence" was officially coined.

(صياغة مصطلح الذكاء الاصطناعي رسميًا)

– Scalability Problems 1966 - 1973
AI faced severe limitations in scaling up; Minsky and Papert published the "Perceptrons" book.

– Knowledge-Based Systems - 1970s
Development of systems relying on domain knowledge, such as the DENDRAL project (Buchanan et al., 1969).

– Expert Systems Boom & AI Winter - 1980s
The industry rapidly grew from millions in 1980 to billions in 1988, with hundreds of companies building expert systems and robots. This bubble burst, leading to the **AI Winter**, where companies failed because they could not deliver on their extravagant promises.

(طفرة النظم الخبيرة ثم "شتاء الذكاء الاصطناعي" بسبب الفشل في الوفاء بالوعود المبالغ فيها)

– Probabilistic & Decision Methods - 1988 - now
A shift towards rigorous mathematical theorems, hard experimental evidence over intuition, and real-world applications over toy examples.

– Now
Massive progress in Machine Learning fueled by probability theory, statistics, and highly specific problem solving. The focus shifted entirely to large datasets. Today, data quality beats algorithmic complexity: a mediocre algorithm with 100 million words of training data easily outperforms the best-known algorithm trained on only 1 million words.

(الاعتماد الكثيف على البيانات الضخمة؛ خوارزمية متوسطة ببيانات ضخمة تتفوق على أفضل خوارزمية ببيانات قليلة)

chapter 3

1. Agent Definition | تعريف الوكيل

An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.

بالعربي: الوكيل هو أي شيء يُنظر إليه على أنه يدرك بيئته من خلال المستشعرات (Sensors) ويؤثر على تلك البيئة من خلال المشغلات (Actuators).

Additional Definition :

A system which has the ability to take a flexible autonomous action in a dynamic, unpredictable and open environment is referred to as an agent.
An agent is a computer system capable of flexible and autonomous action in some environment.

Agent Characteristics:

Characteristic	Meaning	
Flexible	Reactive, proactive, and social	مرن: تفاعلي، استباقي، اجتماعي
Autonomous	Capable of acting independently, exhibiting control over internal state	مستقل: يتصرف باستقلالية ويتحكم في حالته الداخلية

---

2. Agent Examples | أمثلة على الوكلاء

Agent Type	Sensors (المستشعرات)	Actuators (المشغلات)
Human (إنسان)	Eyes, ears, and other organs	Hands, legs, mouth, and other body parts
Robotic (روبوت)	Cameras and infrared range finders	Various motors
Software (برمجي)	Keystrokes, file contents, received network packages	Displays on screen, files, sent network packets

---

3. Agents & Environment | الوكلاء والبيئة

Key Terms:

Term	Definition	
Percept	Perceptual inputs at any given instant	المدخلات الحسية في لحظة معينة
Percept Sequence	Complete history of percepts	التاريخ الكامل للإدراكات
Agent Function	Maps percept sequence to actions: f: P → A*	دالة الوكيل: تربط تسلسل الإدراك بالإجراء
Agent Program	Internal implementation of the agent function, runs on an architecture	برنامج الوكيل: التنفيذ الداخلي

Key Points:

- An agent's choice of action at any given instant can depend on the entire percept sequence observed so far.
- Aim of AI is to build agent programs.
- Agent = Architecture + Program

---

4. Vacuum-Cleaner World Example | مثال عالم المكنسة

Element	Details
Percepts	Location (e.g., A or B) and content (e.g., Dirty or Clean)
Actions	Move left, Move right, Suck, NoOp
Function	If the current square is dirty → Suck; otherwise → move to the other square or standby

---

5. Rational Agents | الوكلاء العقلانيون

Core Definition:

An agent should attempt to do the right thing, based on what it can perceive and the actions it can perform. The right action is the one that will cause the agent to be most successful.

Performance Measure (مقياس الأداء):

An objective criterion for success of an agent's behavior.
Example (Vacuum Cleaner Agent):

- Amount of dirt cleaned up (كمية التراب اللي اتشال)
- Amount of time taken (الوقت المستغرق)
- Amount of electricity consumed (الكهرباء المستهلكة)
- Amount of noise generated (الضوضاء)

Rationality at a Given Time Depends on 4 Factors:

1. The performance measure that defines the criterion of success — مقياس الأداء
2. The agent's prior knowledge of the environment — المعرفة المسبقة بالبيئة
3. The actions that the agent can perform — الإجراءات المتاحة
4. The agent's percept sequence to date — تسلسل الإدراك لحد الان

Rational Agent Definition (Full):

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

---

6. Rationality Exercises

Exercise 1:

An agent is crossing the road and all of a sudden, a heavy thing falls upon him from the sky. Is the agent rational or not? Explain.
Analysis:

- The agent was acting rationally based on its knowledge (crossing the road was the right action for its goal)
- The falling object was unpredictable — the agent couldn't have known
- Rationality ≠ Perfection: the agent maximized expected utility, not actual outcome
- Answer: The agent is rational — it did the right thing based on what it could perceive. The outcome was due to omniscience being impossible (no agent can know the actual outcome of every action).

Exercise 2:

An agent is moving items from one place to another, but fails to move a rather heavy item. Is the agent rational or not? Explain.
Analysis:

- The agent failed to achieve the goal for that item
- But rationality depends on available actions — if the agent couldn't move the heavy item (exceeded its capabilities), it's not irrational
- If the agent could have moved it but chose not to, then it's irrational
- Answer: Depends on whether the action was available to the agent. If the item was beyond its capacity → rational (it did its best). If the agent had the capability but failed → irrational.

Exercise 3:

An agent is designed to speed up mail-sorting task, but one day the agent consumed too much energy/electricity to perform the same task. Is the agent rational or not? Explain.
Analysis:

- The performance measure includes efficiency (energy consumption)
- Consuming too much energy means the agent did not maximize its performance measure
- Answer: The agent is irrational — it failed to maximize the performance measure (which includes energy efficiency). Even if it sorted the mail, the cost was too high..

---

7. Rationality vs. Perfection vs. Omniscience | العقلانية vs العلم المطلق vs الكمال

Concept	Definition	
Rationality	Maximizes expected output	تعظيم الناتج المتوقع
Perfection	Maximizes actual output	تعظيم الناتج الفعلي
Omniscience	Knows the actual outcome of its action	معرفة النتيجة الفعلية (مستحيل)

Key Insight:

- Rationality is distinct from perfection and omniscience
- An omniscient agent knows the actual outcome — impossible in reality
- Rational agents perform actions to modify future percepts and obtain useful information (information gathering, exploration)

Autonomy (الاستقلالية):

An agent is autonomous if its behavior is determined by its own experience — ability to learn and adapt.

بالعربي: الوكيل مستقل لو سلوكه محدد بتجاربه — يعني عنده قدرة يتعلم ويتكيف.

---

8. Irrationality | عدم العقلانية

The same agent can be rational in one circumstance and irrational in another.

Vacuum Agent Example:

Situation	Behavior	Rational?
All dirt is cleaned up, but agent oscillates needlessly back and forth	Keeps moving A→B→A→B with no purpose	Irrational — could be penalized
Better agent: does nothing once sure all squares are clean	Stays idle (NoOp)	Rational
Clean squares can become dirty again	Agent should occasionally check and clean	Rational — exploration
Geography of environment is unknown a priori	Agent should explore rather than stick to A and B	Rational — information gathering

---

Summary | الملخص

Key Definitions:

1. Agent = anything that perceives through sensors and acts through actuators
2. Percept = inputs at a given instant
3. Percept Sequence = complete history of percepts
4. Agent Function = f: P* → A (maps percept sequence to action)
5. Agent Program = implementation of the function
6. Agent = Architecture + Program

Rationality:

- Rational Agent = maximizes expected performance measure
- Depends on 4 factors: performance measure, prior knowledge, available actions, percept sequence
- Rationality ≠ Perfection ≠ Omniscience
- Rationality = expected output; Perfection = actual output; Omniscience = knowing actual outcome (impossible)

Irrationality:

- Same agent can be rational/irrational in different circumstances
- Vacuum agent oscillating after cleaning = irrational
- Better: NoOp when sure clean; explore if unknown

Formulas:

Agent = Architecture + Program
f: P → A*
Rational Agent = Maximize Expected Performance Measure

chapter 4-5

Part 1: Task Environment (PEAS) | بيئة المهمة

The Rule:

To design a rational agent, first we must specify its task environment.

PEAS Model:

Letter	Stands For	
P	Performance measure	مقياس الأداء
E	Environment	البيئة
A	Actuators	المشغلات
S	Sensors	المستشعرات

In designing an agent, the first step must always be to specify the task environment (PEAS) as fully as possible.

PEAS Examples :

1. Auto-Taxi Driver (سواق تاكسي آلي):

PEAS	Details
P	Safe, fast, legal, comfortable trip, maximize profits
E	Roads, other traffic, pedestrians, customers
A	Steering wheel, accelerator, brake, signal, horn
S	Cameras, sonar, speedometer, GPS, engine sensors, keyboard

2. Medical Diagnosis System (نظام تشخيص طبي):

PEAS	Details
P	Healthy patient, minimize costs, satisfaction
E	Patient, hospital, staff
A	Screen display (questions, tests, diagnoses, treatments, referrals)
S	Keyboard (entry of symptoms, findings, patient's answers)

3. Intelligent House (بيت ذكي):

PEAS	Details
P	Optimal room temperature, light on/off when occupants present/absent, optimal power consumption
E	Occupants, house, outside world (including sunlight)
A	Jointed arm and hand to switch on/off buttons
S	Movement sensors, temperature sensors, clock, sound sensor

Part 2: Environment Types | أنواع البيئات

The 6 Dimensions:

1. Fully observable vs. partially observable
2. Deterministic vs. stochastic
3. Episodic vs. sequential
4. Static vs. dynamic
5. Discrete vs. continuous
6. Single agent vs. multi-agent

---

1. Fully Observable (FO) vs. Partially Observable (PO):

An environment is fully observable (FO) when the sensors can detect all aspects that are relevant to the choice of action, otherwise it is partially observable (PO).
Key Points:

- FO environments are more convenient than PO ones
- An environment might be PO because of:
    - Noisy and inaccurate sensors (مستشعرات غير دقيقة)
    - Parts of the state are simply missing from sensor data (أجزاء من الحالة ناقصة)

Examples:

- Vacuum cleaner with local dirt sensor → FO
- Taxi driver → PO

What about chess playing? → Fully Observable (كل القطع ظاهرة)

---

2. Deterministic vs. Stochastic:

The environment is deterministic if the next state is completely determined by the current state and the action executed by the agent.
Key Points:

- If the environment is partially observable → it could appear to be stochastic
- If deterministic except for the actions of other agents → it is strategic (استراتيجي)

Examples:

- Vacuum world → Deterministic
- Taxi driver → Not deterministic (stochastic)

What about chess playing? → Strategic (حتمي بس في لاعب تاني)

---

3. Episodic vs. Sequential:

In an episodic environment, the agent's experience is divided into atomic episodes:

- Each episode consists of the agent perceiving and then performing a single action
- The choice of action in each episode depends only on the episode itself

In a sequential environment, an agent's current action affects future actions.
Examples:

- Mail sorting task → Episodic
- Chess playing → Sequential

What about face recognition / auto-taxi driving? → Face recognition = Episodic; Auto-taxi = Sequential

---

4. Static vs. Dynamic:

A static environment is unchanged while an agent is deliberating, while dynamic environment may change.
Key Points:

- Static environments are easy to deal with
- Semi-dynamic: the environment itself does not change with the passage of time but the agent's performance score does

Examples:

- Taxi driving → Dynamic
- Chess when played with a clock → Semi-dynamic
- Crossword puzzles → Static

What about chess playing (without clock)? → Static (البيئة ثابتة أثناء التفكير)

---

5. Discrete vs. Continuous:

A discrete environment has a finite and clearly defined states, percepts and actions.
Examples:

- Chess → Discrete (finite number of discrete states, percepts, actions)
- Taxi driving → Continuous (continuous states and actions)

---

6. Single Agent vs. Multi-Agent:

In a single agent environment, an agent operates by itself. In a multi-agent environment, more than one agent act.
Examples:

- Crossword → Single agent
- Chess → Two agents (multi-agent)

Summary

The environment type largely determines the agent design.
The real world is: partially observable, stochastic, sequential, dynamic, continuous, and multi-agent.
Building an autonomous agent in such complex environments is a hard AI task.

Part 3: Agent Implementation — Program | تنفيذ الوكيل

Agent = Architecture + Program

Agent Program Skeleton:

All agent programs can have the same skeleton:

- Input = current percepts
- Output = action
- Program = manipulates input to produce output

The Simplest Solution — Lookup Table (الجدول الجدولي):

Keep "entire percept sequence" in the memory. Map it to the "action" table (Lookup).
Problems (مشاكل):

- Simple to build, but practically infeasible (بسيط بس غير عملي)
- Space to store the table (مساحة تخزين ضخمة)
- Take a long time to build the table (وقت طويل لبناء الجدول)
- No autonomy (لا يوجد استقلالية)
- Learning could be very time consuming (التعلم ياخد وقت طويل)

Example: Playing chess needs 35^100 entries — رقم ضخم مستحيل.

Part 4: Agent Types | أنواع الوكلاء

Rather than a table, we can produce rational behavior from a small piece of code.
Four basic types:

1. Simple reflex agents
2. Model-based reflex agents
3. Goal-based agents
4. Utility-based agents

(+ Learning agents)

Type 1: Simple Reflex Agents (الوكيل الانعكاسي البسيط)

Select actions on the basis of the current percept, ignoring the rest of the percept history — i.e., Condition-Action Rule.
Vacuum Example:
if status == Dirty then return Suck
else if location == A then return Right
else if location == B then return Left
Limitations (عيوب):

- Simple, but very limited intelligence — بسيط بس ذكاء محدود جداً
- Works only if the correct decision can be made on the basis of the current percept → environment must be fully observable
- Infinite loops are often unavoidable → Escape possible by randomizing

---

Type 2: Model-Based Reflex Agents (الوكيل الانعكاسي القائم على نموذج)

The agent should keep track of the part of the world it can't see now.
Maintain an internal state that depends on the percept history.
Requires two kinds of knowledge:

1. How the world evolves itself — كيف العالم بيتطور لوحده
2. How the agent's own actions affect the world — كيف إجراءات الوكيل بتأثر على العالم

Needs to maintain a world model (نموذج للعالم).
بالعربي: بيحتفظ بحالة داخلية عشان يتتبع اللي مو شايفه. محتاج يعرف كيف العالم بيتطور وكيف أفعاله بتأثر

Type 3: Goal-Based Agents (الوكيل القائم على الهدف)

Knowing the current state is not always enough to decide what to do (e.g., decision at a road junction).
The agent needs goal information that describes situations that are desirable.
Combine goal info with results of possible actions → choose actions that achieve the goal.
Usually requires search and planning.
بالعربي: معرفة الحالة الحالية مو كافي — يحتاج هدف. يخطط وبيبحث عشان يوصل للهدف.

Reflex Agent vs. Goal-Based Agent:

Aspect	Reflex Agent	Goal-Based Agent
Efficiency	More efficient	Less efficient
Flexibility	Low — must rewrite rules	More flexible — knowledge represented explicitly and can be modified
Behavior change	Rewrite many condition-action rules	Easily changed
Future	Doesn't consider	Takes future into account

بالعربي: الـ Goal-Based أقل كفاءة بس أكثر مرونة — تعديل الهدف سهل، وياخد المستقبل في الحسبان.

Type 4: Utility-Based Agents (الوكيل القائم على المنفعة)

Goals alone are not enough to generate high-quality behavior in most environments.

- What if multiple roads lead to the destination from a junction?
- Which road is better — safe, cheap, fast...?

Key Concepts:

- If state A is preferred over state B → A has a higher utility
- A utility function maps a state onto a real number describing the degree of happiness/satisfaction
- Happy-unhappy vs. how much happy

بالعربي: الهدف لوحده مو كفاية — لو في طرق كثيرة، يحتاج يفضل. دالة المنفعة بتحول الحالة لرقم يوصف درجة السعادة/الرضا.

Type 5: Learning Agents (الوكيل المتعلم)

Turing: Instead of actually programming intelligent machines by hand (too much work), build learning machines and then teach them.
Learning allows the agent to operate in initially unknown environments and become more competent than its initial knowledge alone might allow.
Components (المكونات):

Component	Responsibility	
Learning Element	Introduces improvements in performance element; uses feedback from the critic on how the agent is doing and how the performance element should be modified	عنصر التعلم — يسوي تحسينات بناءً على تغذية الناقد
Performance Element	Responsible for selecting external actions	عنصر الأداء — يختار الإجراءات
Critic	Provides feedback on how well the agent is doing	الناقد — يدي تغذية راجعة
Problem Generator	Suggests actions that lead to new and informative experiences (explore vs. exploit)	مولّد المشاكل — يقترح استكشاف

بالعربي: الوكيل المتعلم يتحسن مع الوقت. عنده عنصر يتعلم، عنصر يشتغل، ناقد يقيّم، ومولّد مشاكل يقترح استكشاف. التوازن بين explore (استكشاف) و exploit (استغلال)

Summary

1. Agents interact with environments through actuators and sensors
2. Agent functions describe what the agent does, and agent programs implement the function
3. The performance measure evaluates the behavior of the agent
4. A rational agent maximizes expected performance
5. PEAS descriptions define task environments
6. Environments are categorized along several dimensions: observable, deterministic, static, (...)
7. Several basic agent architectures exist:
    - reflex
    - model-based reflex
    - goal-based
    - utility-based

chapter 6 - 7

Part 1: Fundamental Concepts | المفاهيم الأساسية

Previous chapters:

- Agent Definition: computer system capable of flexible and autonomous action in some environment
- Rationality: select action expected to maximize performance measure
- PEAS: Performance, Environment, Actuators, Sensors
- Environment types: Fully Observable, Deterministic, Episodic, Static, Discrete, Single
- Agent types: Simple Reflex, Model Based, Goal Based, Utility Based, Learning

---

Example — The Fox, Goose, and Oats Problem:

A man is travelling to the market with a fox, a goose, and a bag of oats. He comes to a river. The only way across is a boat that can hold the man and exactly one of the fox, goose or bag of oats. The fox will eat the goose if left alone with it, and the goose will eat the oats if left alone with it.
Question: In how many times can the man get all his possessions safely across the river?

Answer: 7 trips (the classic solution).
The Real Question :
How can we get a machine to solve all problems like this automatically?
We need a systematic approach that considers all possibilities.
Search problems define the possibilities, and search algorithms explore these possibilities.

Solving Problems — Introduction:

Search is one of the most powerful approaches to problem solving in AI:

- Systematically explores the alternatives (يستكشف البدائل بشكل منظم)
- Finds the sequence of steps (path) towards a solution (يلاقي تسلسل الخطوات نحو الحل)

Problem Space Hypothesis (فرضية فضاء المشكلة):

- All goal-oriented symbolic activities occur in a problem space
- Search in a problem space is claimed to be a completely general model of intelligence

بالعربي: كل الأنشطة الرمزية الموجهة نحو الهدف بتصير في فضاء مشكلة. والبحث في فضاء المشكلة يُعتبر نموذج عام للذكاء.

Key Idea:

Many problems can be viewed as reaching a goal state from a given starting point / initial state.

- Often there is an underlying state space that defines the problem and its possible solutions formally
- The space can be traversed by applying a successor function (operators) to proceed from one state to the next
- If possible, information about the specific problem or general domain is used to improve the search → informed search:
    - Experience from previous instances
    - Strategies expressed as heuristics
    - Simpler versions of the problem
    - Constraints on certain aspects

بالعربي: المشاكل ممكن تُرى كالوصول من حالة ابتدائية لحالة هدف. في فضاء حالة (State Space) يتحدد المشكلة وحلولها.
نتحرك في الفضاء باستخدام successor function. ولو استخدمنا معلومات إضافية → informed search (بحث موجه).

Part 2: Problem Formulation | صياغة المشكلة

The 6 Components (مكونات المشكلة):

Component	Meaning	
Initial state	The agent starts in — i.e., s0	الحالة الابتدائية
Goal State	Final state — i.e., sn	حالة الهدف
Operators / Actions	That change the state of the world — i.e., Action(s)	العمليات/الإجراءات
State Space	Set of all states reachable from initial state by any sequence of actions	فضاء الحالة
Goal test	To determine if it is the goal state	اختبار الهدف
Path cost	When it matters how we get there	تكلفة المسار

State Space Details:

Together, the initial state, actions, and transition model implicitly define the state space of the problem.

- A path is any sequence of actions that lead from one state to another
- Directed networks, Graphs, Trees etc.

Part 3: Example Problems | أمثلة على المشاكل

1. Vacuum Cleaner World (عالم المكنسة):

Component	Details
States	Dirt & Vacuum Location
Initial state	$2 \times 2^2 = 8$ possible states; any of them
Operators / Actions	Suck(S), Left(L), Right(R)
Transition Model	Description of effect of action
Goal test	All squares are clean
Path cost	Number of steps

---

2. n-Queens Problem (مشكلة n ملكات):

The problem is to place n queens on an n-by-n chessboard so that no two queens attack each other by being in the same row, same column, or same diagonal.

Solution Approach:

Since each of the four queens has to be placed in its own row, all we need to do is to assign a column for each queen on the board.
Steps:

1. Start with empty board, place Queen 1 at (1,1)
2. Place Queen 2 — try column 1 (same column → no), column 2 (diagonal → no), then (2,3) ✓
3. Queen 3 — no acceptable position → dead end → backtrack
4. Backtrack Queen 2 to (2,4)
5. Queen 3 at (3,2) → another dead end for Queen 4 → backtrack all the way to Queen 1
6. Queen 1 to (1,2)
7. Queen 2 to (2,4)
8. Queen 3 to (3,1)
9. Queen 4 to (4,3) → Solution!

Key concept: Backtracking

8-Queens Formulation :

Component	Details
States	Any arrangement of n ≤ 8 queens
Initial state	No queen on the board
Operators / Actions	Add queen to any empty square; OR add queen to leftmost empty square such that it is not attacked by other queens
Transition Model	Returns the board with a queen added to the specified square
Goal test	8 queens on the board, none attacked
Path cost	Time or how many moves it made

---

3. Sliding Tile Puzzle (8 — لغز البلاط المنزلق):

Component	Details
States	Location of each of the eight tiles and the blank in one of the nine squares
Initial state	Any state can be designated as the initial state
Operators / Actions	Movements of the blank space: Left, Right, Up, Down
Transition Model	A tile will be swapped with the blank
Goal test	Checks whether the state matches the goal configuration
Path cost	Number of steps in the path

بالعربي: الحالة = مكان كل البلاطات والفراغ. الحركات = تحريك الفراغ. الهدف = مطابقة التكوين المطلوب. التكلفة = عدد الخطوات.

---

4. Travelling in Romania (السفر في رومانيا):

Component	Details
States	Any city on the map
Initial state	Starting city e.g. "at Arad"
Operators / Actions	Actions(s) = set of actions available in state s
Transition Model	Results(s,a) = state that results from action a in state s
Goal test	s = "at Bucharest"
Path cost	Sum of distances, number of actions executed, etc. — c(x,a,y) is the step cost, assumed to be ≥ 0

Part 4: World State vs. Search State | حالة العالم vs. حالة البحث

Aspect	World State (حالة العالم)	Search State (حالة البحث)
Definition	Complete and detailed description of the environment	Simplified representation focusing only on elements relevant to the problem
Scope	Comprehensive; includes all aspects, relevant or not	Focused; only aspects necessary for solving the problem
Components	Agent's position, ghosts' positions, food dots, agent's orientation, any other environmental details	Agent's position, relevant items (e.g., dots), may exclude irrelevant details like ghost positions or agent orientation
Purpose	Describe the entire environment as it exists at a moment in time	Abstract the environment for more efficient problem-solving and search
Complexity	More complex due to inclusion of all details	Less complex due to focus on relevant aspects only
Example	Pac-Man at (3,2), Ghosts at (1,1) and (5,5), 5 dots remaining, Pac-Man facing East	Pac-Man at (3,2), 5 dots remaining
Use Case	Understanding full context of the environment	Used by search algorithms to efficiently explore and solve the problem

بالعربي:

- World State: كل تفاصيل البيئة (زي Pac-Man مع مواقع الأشباح واتجاه الوجه)
- Search State: الملخص اللي يهمنا بس (مكان Pac-Man والنقط المتبقية)

Part 5: State Space Graphs vs. Search Trees | فضاء الحالة vs. شجرة البحث

State Space Graph (رسم فضاء الحالة):

Mathematical representation of a search problem:

- Vertices are states (world configurations)
- Edges correspond to successors
- The goal test is a set of goal nodes
- In a search graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea

---

Search Tree (شجرة البحث):

A search tree:

- Start state is the root of the tree
- Children are successors
- A plan is a path in the tree
- For most problems, we do not actually generate the entire tree

Key Difference:

	State Space Graph	Search Tree
Nodes	Each state once	Same state can appear multiple times (paths)
Structure	Graph (can have cycles)	Tree (no cycles)
Purpose	Mathematical model	What the algorithm actually explores

---

Part 6: Search Terminology | مصطلحات البحث

1. Expansion of Nodes (توسيع العقد):

As states are explored, the corresponding nodes are expanded by legal operators (actions).

2. The Fringe / Frontier (الحدود):

The fringe (frontier) is the set of nodes that are newly generated and not yet visited.

- Newly generated nodes are added to the fringe.

3. Search Strategy (استراتيجية البحث):

Determines the selection of the next node to be expanded.

- Can be achieved by ordering the nodes in the fringe.

Summary

1. Search = systematic exploration of alternatives to find a path to solution
2. Problem Formulation has 6 components: Initial state, Goal state, Operators, State Space, Goal test, Path cost
3. State Space = all states reachable from initial state
4. World State = all details; Search State = only relevant details
5. State Space Graph = each state once; Search Tree = paths (states can repeat)
6. Fringe = newly generated nodes not yet visited
7. Search Strategy = determines which node to expand next (ordering the fringe)

The 4 Example Problems:

Problem	States	Actions	Goal
Vacuum World	Dirt + Location (8 states)	Suck, Left, Right	All clean
8-Queens	Arrangement of queens	Add queen (not attacked)	8 queens, none attacked
Sliding Tile	Tile locations + blank	Move blank L/R/U/D	Match goal config
Romania	City	Move to adjacent city	At Bucharest

---

chapter 8 and 9

Part 1: General Search Algorithms | خوارزميات البحث العامة

1. TREE-SEARCH Algorithm :

```
function TREE_SEARCH(problem) returns a solution, or failure
    initialize the frontier as a specific work list (stack, queue, priority queue)
    add initial state of problem to frontier
    loop do
        if the frontier is empty then return failure
        choose a node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        for each resulting child from node
            add child to the frontier
```

بالعربي:

1. حط الحالة الابتدائية في الـ frontier
2. لو الـ frontier فاضي ← فشل
3. اختار عقدة واشيلها من الـ frontier
4. لو دي الحالة الهدف ← ارجع الحل
5. وإلا ← وسّعها (ولّد أبناءها) وضيفهم للـ frontier

---

2. GRAPH-SEARCH Algorithm:

```
function GRAPH_SEARCH(problem) returns a solution, or failure
    initialize the explored set to be empty
    initialize the frontier as a specific work list (stack, queue, priority queue)
    add initial state of problem to frontier
    loop do
        if the frontier is empty then return failure
        choose a node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        add the node state to the explored set
        for each resulting child from node
            if the child state is not already in the frontier or explored set then
                add child to the frontier
```

Key Difference:

	TREE-SEARCH	GRAPH-SEARCH
Explored set	❌ لا	✅ نعم
Repeated states	ممكن يزور نفس الحالة	ما يزور نفس الحالة
Memory	أقل	اكثر

Part 2: Search Algorithm Properties | خصائص خوارزميات البحث

Node Infrastructure :

Each node "n" of the tree/graph has:

- n.State — to which the node corresponds (الحالة)
- n.Parent — the node in search tree that generated it (الأب)
- n.Action — the action applied to the parent to generate it (الإجراء)
- n.Path-Cost — cost from initial state to the node (تكلفة المسار)

Performance Measurement (4 Properties):

Property	Question	
Completeness	Is the algorithm guaranteed to find a solution when there is one?	هل يضمن يلاقي حل لو موجود؟
Time complexity	How long does it take to find the solution?	ياخد وقت قد إيش؟
Space complexity	How much memory does it need?	محتاج ذاكرة قد إيش؟
Optimality	If more than one solution, does it find the least-cost solution?	لو في أكتر من حل، يلاقي الأقل تكلفة؟

Tree Search Complexity :

Notation:

- b = maximum branching factor (أقصى عدد أبناء)
- m = maximum depth of the state space (may be ∞) (أقصى عمق)
- d = depth of the least-cost solution (عمق الحل الأقل تكلفة)

Number of nodes in entire tree:

$1 + b + b^2 + ... + b^m = O(b^m)$

بالعربي: عدد العقد في الشجرة كلها = O(b^m) — لأن كل مستوى بيضرب في b.

Example:

- b = 3 (in the tree)
- m = 3 (assuming root at depth 0)
- d = 2 (path 1-6-7 is least-cost solution)

We are often interested in worst-case complexity.

Part 3: Search Strategies Overview | نظرة عامة على استراتيجيات البحث

Two Main Categories:

Type	Also Called	Uses Info?	
Uninformed Search	Blind, exhaustive, brute-force	❌ No info about problem	بحث أعمى — ما يستخدم معلومات
Informed Search	Heuristic, intelligent	✅ Uses info to guide search	بحث موجه — يستخدم معلومات

Uninformed: May be quite inefficient (ممكن يكون غير كفء).
Informed: Usually guesses the distance to goal — but not always possible (مو دايماً ممكن).

Uninformed Search Strategies (5):

1. Depth-first search (DFS)
2. Breadth-first search (BFS)
3. Uniform-cost search (UCS)
4. Depth-limited search (DLS)
5. Iterative deepening search (IDS)

Part 4: Depth-First Search (DFS) | البحث بالعمق أولاً

Idea: Explore as deep as possible along each branch before backtracking.

How it works:

- Uses a stack (LIFO — Last In First Out) for the frontier
- Expands the deepest node first
- **Note**: "The search ends when the goal node is generated (not expanded)"

DFS Properties:

- Complete? ❌ No (in infinite spaces) — ممكن يضيع في فرع لا نهائي
- Optimal? ❌ No — ممكن يلاقي حل مو الأقل تكلفة
- Time O(b^m) — ممكن يكون أسوأ من BFS
- Space O(bm) — linear! (ميزة كبيرة)

Part 5: Breadth-First Search (BFS) | البحث بالعرض أولاً

Idea: Explore all nodes at the current depth before going deeper.

How it works:

- Uses a queue (FIFO — First In First Out) for the frontier
- Expands the shallowest node first
- **Note**: "The search ends when the goal node is generated (not expanded)"

BFS Algorithm (Informal):

1. Form one-element List consisting of the root node
2. While the List is not empty or the goal has been reached:
    - a. If the first element is the Goal node → return SUCCESS
    - b. If not → remove first element from front, add its children to the BACK of the List
3. Return FAILURE

BFS Properties:

Property	Value	
Complete?	✅ Yes (if d is finite)	لو العمق محدود
Optimal?	✅ Yes — only if costs are all the same	أمثل — بس لو التكاليف متساوية
Time	O(b^d)	وقت أسّي
Space	O(b^d) — big problem!	مساحة أسّية — مشكلة كبيرة

DFS vs. BFS — Comparison

Aspect	DFS	BFS
Data structure	Stack (LIFO)	Queue (FIFO)
Expands	Deepest node	Shallowest node
Complete?	❌ No	✅ Yes (finite d)
Optimal?	❌ No	✅ Yes (equal costs)
Time	O(b^m)	O(b^d)
Space	O(bm) — linear ✅	O(b^d) — exponential ❌

Part 6: Uniform-Cost Search (UCS) | البحث بالتكلفة الموحدة

Idea:Similar to BFS, but takes path cost into account — an improved version of BFS.

- BFS finds the shallowest goal state
- UCS modifies BFS by expanding the lowest cost node first

Key Points:

- Path cost measured by a cost function
- Nodes ordered in a priority queue (طابور أولوية)
- UCS = BFS if all step costs are equal (very rare)
- **Note:** "The search ends when the goal node is **expanded** (not generated)"
    - DFS/BFS: ends when goal is generated
    - UCS: ends when goal is expanded — Why? Because a cheaper path might still be found!

UCS Properties:

Property	Value	
Complete?	✅ Yes (if best solution has finite cost and min arc cost is positive)	بشروط
Optimal?	✅ Yes! (proof in the upcoming texts via A*)	أمثل
Time	O(b^(C*/E)) — exponential in effective depth	وقت أسّي
Space	O(b^(C*/E))	مساحة أسّية

Where:

- C* = cost of cheapest solution
- E = minimum arc cost
- C*/E = "effective depth" (العمق الفعّال)

UCS Issues:

- UCS explores increasing cost contours (بيستكشف كونتورات تكلفة متزايدة)
- Explores options in every "direction" (بيستكشف في كل الاتجاهات)
- No information about goal location (مافي معلومات عن مكان الهدف)

Part 7: Depth-Limited Search (DLS) | البحث المحدود العمق

Idea: DFS with a depth limit (l) — عشان نتجنب مشكلة الفرع اللانهائي في DFS.

Note: Slides mention DLS algorithm but don't show full details — it's DFS with a limit.

Part 8: Iterative Deepening Search (IDS) | البحث بالتعميق التكراري

Idea: IDS performs a series of DFS with increasing depth limits — exploring deeper each time.

How it works:

1. Run DFS with depth limit 1. If no solution...
2. Run DFS with depth limit 2. If no solution...
3. Run DFS with depth limit 3. ...

Isn't that wastefully redundant?
Generally most work happens in the lowest level searched, so not so bad!
بالعربي: رغم إنه يكرر، معظم الشغل يصير في المستوى الأخير — فالتكرار مو مشكلة كبيرة.

IDS Properties:

Property	Value
Complete?	✅ Yes
Time complexity	O(b^d) (exponential) — same as BFS
Space complexity	O(bd) (linear) — same as DFS
Optimality	✅ Yes (when all step costs are equal)

Key Insight: IDS combines advantages of DFS and BFS:

- Memory: like DFS → O(bd) — modest
- Time/optimality: like BFS → O(b^d) — complete and optimal

IDS Time Complexity:

$N(IDS) = (d)b^1 + (d-1)b^2 + (d-2)b^3 + ... + b^d$
Where: b = branching factor, d = depth of solution
Example (b=10, d=5):

- N(IDS) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450
- N(BFS) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110

Conclusion: Although IDS revisits nodes, time complexity remains O(b^d) — asymptotically the same as BFS.

IDS Advantages:

- Memory Efficiency: less memory than BFS (behaves like DFS)
- Optimality: finds optimal solution when all step costs are equal
- Completeness: always finds a solution if one exists
- No depth knowledge required: automatically adjusts depth limits

IDS Disadvantages:

- Repetition: revisits nodes at shallower depths — but overhead is small
- Not Efficient with Unequal Costs: not optimal if step costs vary

---

Part 9: Comparison Table | جدول المقارنة

Algorithm	Data Structure	Complete?	Optimal?	Time	Space
DFS	Stack	❌ No	❌ No	O(b^m)	O(bm) ✅
BFS	Queue	✅ Yes	✅ Yes (equal costs)	O(b^d)	O(b^d)
UCS	Priority Queue	✅ Yes	✅ Yes	O(b^(C*/E))	O(b^(C*/E))
DLS	Stack	❌ No (if l < d)	❌ No	O(b^l)	O(bl)
IDS	Stack	✅ Yes	✅ Yes (equal costs)	O(b^d)	O(bd) ✅
bidirectional (if applicable)	don't know	yes	yes	O(b^(d/2))	O(b^(d/2))

- DFS: space-efficient but incomplete/not optimal
- BFS: complete/optimal but huge space
- UCS: optimal (any cost) but slow
- IDS: best of both worlds — complete, optimal, linear space

Part 10: Bidirectional Search

Idea:Search forward from start and backward from goal simultaneously — meet in the middle.

Note: Slides mention it as a students' exercise — not detailed

Summary

Formulas:

Tree nodes: 1 + b + b² + ... + b^m = O(b^m)
BFS time/space: O(b^d)
DFS space: O(bm)
UCS time/space: O(b^(C*/ε))
IDS time: O(b^d); space: O(bd)

Key Distinctions:

	DFS	BFS	UCS	IDS
Expands	Deepest	Shallowest	Lowest cost	DFS with limits
Ends when goal is	Generated	Generated	Expanded	Generated
Space	Linear ✅	Exponential ❌	Exponential	Linear ✅
Complete	No	Yes	Yes	Yes
Optimal	No	Equal costs	Yes	Equal costs

The important Ideas:

1. Tree-Search vs. Graph-Search: Graph has explored set (no repeated states)
2. DFS: space-efficient but incomplete
3. BFS: complete but space-hungry
4. UCS: optimal with any costs — but slow
5. IDS: combines DFS space + BFS completeness — best general choice

chapter 10 and 11

Part 1: Informed Search — Background | خلفية البحث الموجه

Why Informed Search?

Domain-specific information can improve the search process by:

- Guiding the algorithm towards goal state (توجيه الخوارزمية نحو الهدف)
- Pruning off the states that do not lie on a solution path (إزالة الحالات اللي مو على مسار الحل)

What Does It Require?

In addition to start state, successor functions, and goal state, it requires a function for evaluating states.

- This function is called a heuristic function.
- It estimates the cost of the optimal path from a state to the goal state (cf. uniform-cost search).

Key difference: Uniform-cost search uses actual path cost; heuristic uses estimated cost.

Part 2: What is a Heuristic? | ما هي الحدسية؟

Definition:

A heuristic is a rule of thumb, or educated guess that reduces or limits the search for solutions in domains that are difficult and poorly understood.

بالعربي: الحدسية هي قاعدة إبهام أو تخمين مبني على معرفة — تقلل أو تحدد البحث في مجالات صعبة وغير مفهومة كويس.

Unlike algorithms, heuristics do not guarantee feasible solutions and are often used with no theoretical guarantee.

Example — Greedy Heuristic:

A shop with several checkouts, including express checkout.
Greedy Heuristic: It is usually best to go to the one with the shortest queue.
But... :

- Shortest queue — no customer at the checkout? → No cashier → Machines not functioning
- Shortest queue — only one person at the checkout? → had three trolleys full of shopping
- Longest queue — at the express-checkout many people? → each person holds few items

When Are Heuristics Used? :

- There are no mathematical techniques available to find an optimal solution and a good solution is enough
- The search space is so large that exhaustive search techniques are too costly
- If search space is small and exact algorithm is available → heuristics might not be preferred

Part 3: Heuristic Function — Characteristics | خصائص دالة الحدسية

Definition:

A heuristic function h : S → R, where S is a set of all states and R is a set of real numbers, maps each state s in the state space S into a measurement h(s) which is an estimate of the relative cost.

Conditions:

Condition	Meaning
h(n) = 0	For the goal state
h(n) = ∞	For a dead-end state
h(n) > 0	For all other states

Part 4: Heuristic Examples | أمثلة على الحدسيات

1. Sliding Tiles Puzzle (8-puzzle):

In total, there are 9! = 362,880 possible states.
With a good heuristic, it is possible to reduce this search space to less than 50 states.

Heuristic $h_1(n)$ — Number of Misplaced Tiles:

Calculate number of misplaced tiles.
Example: $h_1(n) = 7$
Problem :
The number of misplaced tiles gives a rough indication of how far we are from the goal.
It does not tell us how badly misplaced tiles are.
$h_1(A) = 3$ & $h_1(B) = 3$ — May or may not require equal number of moves to reach the goal.

Heuristic $h_2(n)$ — Manhattan Distance:

How far is the misplaced tile from the goal position?
Calculate vertical & horizontal distance of each misplaced tile.
Tile: (r_1, c_1); Goal Position: (r_2, c_2)
$MD = |r_1 - r_2| + |c_1 - c_2|$
"Calculate sum of Manhattan distances of all tiles from their goal positions"
Example:

Tile	Position	Goal	$\lvert r_1-r_2\rvert + \lvert c_1-c_2\rvert$	MD
5	(1,1)	(3,3)	|3-1| + |3-1|	4
4	(1,2)	(2,3)	|2-1| + |3-2|	2
6	(2,1)	(3,2)	|3-2| + |2-1|	2
1	(1,1)	(2,2)	|2-1| + |2-1|	2
8	(2,3)	(2,1)	|2-2| + |1-3|	2
7	(3,1)	(3,1)	|3-3| + |1-1|	0
3	(3,2)	(1,3)	|1-3| + |3-2|	3
2	(3,3)	(1,2)	|1-3| + |2-3|	3

$h_2(n) = 4+2+2+2+2+0+3+3 = 18$

Summary — h_1 vs. h_2:

Aspect	$h_1(n)$	$h_2(n)$
Tells	Only how many tiles are misplaced	How badly tiles are misplaced
Smaller value	Closer to goal	Closer to goal
Information	Additional/correct info → admissible	More info than h_1 → elegant
Effect	Guides which state to visit next — does NOT reduce possible states	Same

Both give additional/correct information which leads to goal achievement quickly i.e., admissible.

It will not actually reduce the possible states but guide which state should be visited next preferably.

$h_2(n)$ is elegant and provides more information than $h_1(n)$.

---

2. Pancake Problem (مشكلة البان كيك):

Imagine pancakes of different sizes stacked on a plate. There are four pancakes.
Goal state from top to bottom: [1,2,3,4] — 1 is smallest, 4 is largest.
Possible arrangements: 4! = n!
Only action allowed: Flip top k pancakes at a time.

Heuristic Function:

Calculate number of pancakes not ordered correctly.
Start checking from the bottom upward.
h(n) = n − number of consecutively correct ordered pancakes
Example:

- Start State → h(n) = 3
- Goal State → h(n) = 0

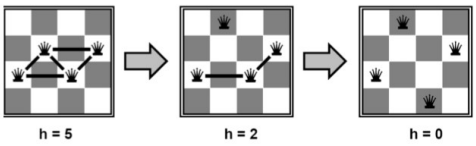
---

3. Solving Maze (حل المتاهة):

How to compute h(n)?
1. Euclidean Distance: $\sqrt{((x_2-x_1)^2 + (y_2-y_1)^2)}$
2. Manhattan Distance: $|x_2-x_1| + |y_2-y_1|$

---

4. 4-Queens Problem:



Part 5: Informed Search Algorithms | خوارزميات البحث الموجه

Categories:

Category	Algorithms	Improvement Over
Iterative Improvement / Local Search	Hill Climbing	DFS
	Beam Search	BFS
Best First Search		—
A Search*		—

بالعربي:

- Hill Climbing = تحسين لـ DFS
- Beam Search = تحسين لـ BFS
- Best First Search و A* = خوارزميات بحث موجه

Part 6: Hill Climbing Search | البحث بالتسلق

Introduction :

Same as depth-first strategy, except the node expansion takes heuristic value into account.
Continually moves uphill:

- Always look for the next change which will improve the current state
- Gradient Descent Search is a variation that moves downhill

Key Characteristics:

- Very simple strategy and requires low space
- Stores only the state and its evaluation — no search tree is maintained
- Also referred as Local Search Algorithm

Problems :

- Local maxima / minima (القمم/القيعان المحلية)
- Search method can't go higher, but is not at a satisfactory solution

بالعربي: المشكلة الأساسية: ممكن يعلق في قمة محلية — مو أحسن حل ممكن، بس مو قادر يتحرك لأنه مو شايف حاجة أحسن حوليه.

Hill Climbing — Algorithm :

```
path = [start];
state = start
while state ≠ goal_state {
   X = generate_children(state)  // X is a list
   state = best(X)
   add state to path
}
return path
```

Hill Climbing — Problem Example:

Given a network of cities connected by roads, find a path from start city S to goal city G using Hill-Climbing Search.
Heuristic Function: Straight-line distance from city n to goal city G (Manhattan or Euclidean distance).

City	h(n)
S	11.0
A	10.4
B	6.7
C	4.0
D	8.9
E	6.9
F	3.0
G	0

Tracing: (Hill Climbing follows the best neighbor at each step)
Example trace:

- S (11.0) → best neighbor is A (10.4) → best neighbor is B (6.7) → ... → G (0)

But watch out: Hill Climbing might get stuck at a local minimum — a city where all neighbors have higher h, but it's not G.

Hill Climbing — Summary

- Looks one step ahead to determine if any successor is better than current state
    - If there is such a successor → move to that one
    - Otherwise halt in the same state
- Does not allow backtracking or jumping to an alternative path since it doesn't remember where it has been
- Not complete since the search could terminate at local minima/maxima

Summary

1. Informed Search uses heuristic functions to guide search
2. Heuristic = rule of thumb / educated guess — does NOT guarantee solutions
3. Heuristic function h(n) estimates cost from state to goal:
    - h(goal) = 0
    - h(dead-end) = ∞
    - h(other) > 0
4. Admissible = doesn't overestimate the true cost
5. Hill Climbing = DFS + heuristic — local search, low space, not complete (local maxima)
6. Beam Search = BFS + heuristic (improvement over BFS)
7. Best First Search and A* = informed search algorithms

Heuristic Examples:

Problem	Heuristic
8-puzzle	h_1 = misplaced tiles; h_2 = Manhattan distance
Pancake	h(n) = n − correctly ordered from bottom
Maze	Euclidean or Manhattan distance to goal
4-Queens	Number of attacking queen pairs

Hill Climbing Key Points:

- Same as DFS but uses heuristic value
- Moves uphill (to better state)
- Low space — only current state + evaluation
- No search tree maintained
- Problems: local maxima/minima
- Not complete — can't backtrack
- Algorithm: generate children → pick best → repeat until goal or no improvement

chapter 12

Part 1: Beam Search | البحث بالحزمة

Variation of breadth-first search

- Limit the queue to the best w nodes
    - w is the beam width — keep it constant throughout the search process
- Expand only those w nodes, and throw away all the other nodes (even the node is on the solution path!)
    - Select the best w of the expanded nodes (cf. genetic algorithms — survival of fittest)
- May not produce a solution, even the solution exists — not complete

Beam Search هي نسخة من BFS بس تحتفظ بـ w عقدة بس (أفضل w). الـ w اسمها beam width وثابتة طول البحث
بتوسّع الـ w بس وبترمي الباقي — حتى لو كان على مسار الحل! عشان كذا مو مكتملة — و ممكن ما تلاقي حل حتى لو موجود

Key Points:

Aspect	Detail
Based on	BFS (breadth-first search)
Limit	Only w best nodes in queue
w	Beam width — constant throughout
Discards	All other nodes (even solution-path nodes!)
Complete?	❌ No — may not find a solution even if one exists
Analogy	Genetic algorithms — survival of the fittest

Beam Search — Problem Example:

Consider an AI-assisted project planning problem: a project is developed through several stages (requirements analysis, design, implementation, resource allocation, testing, deployment). Each node represents a different partial project plan.
Using Beam Search with beam width w = 2, retain at each level only the two partial plans having the smallest estimated remaining cost, and continue until a complete project plan represented by goal state G is reached.
Heuristic Function h(n): Estimated cost required to complete the project from state n:

- Labor Cost (تكلفة العمالة)
- Resources Cost (تكلفة الموارد)
- Delay Cost (تكلفة التأخير)
- Risk Cost etc. (تكلفة المخاطر)

Example table:

Stages	h(n)
S	—
A	10.4, ∞, —
B	6.7, 3, 10
C	4, 7, 5, ∞
D	2, 8.9, —
E	6.9, 8, 5
F	9, 2, 3
G	0

Tracing idea: At each level, keep only the best w = 2 nodes (smallest h). Continue until G.

Beam Search — Algorithm:

```
Q = [start];
X = ϕ
while  (1) {
        while (!Q.isEmpty()) {
          state = Q.dequeue()
          if (state == goal_state)
            return 1
          X = X ∪ generate_children(state)
        }
        Q = select_best(w, X)
        X = ϕ
        if Q.isEmpty()    // search-space exhausted
          return 0
}
```

Part 2: Best-First Search (BestFS)

Introduction:

A combination of depth-first (DFS) and breadth-first search (BFS):

- DFS is good because a solution can be found without computing all nodes
- BFS is good because it does not get trapped in dead ends
- Best First Search (BestFS) allows us to switch between paths thus gaining the benefits of both approaches

Core Idea:

Best-first search expands the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly.
Best-first search evaluates nodes by using just the heuristic function.
This means it uses h(n) as the evaluation function f(n) i.e., f(n) = h(n).

- For Romania problem: h_SLD(n) = straight-line distance from n to Bucharest

Beam Search vs. BestFS :

Aspect	Beam Search	Best-First Search
Based on	BFS	DFS + BFS combination
Queue	Only w best nodes	All nodes (priority by h)
Complete?	❌ No (throws away nodes)	❌ No (can loop without repeated-state checking)
Optimal?	❌ No	❌ No
Memory	O(w) — very small	O(b^m) — keeps all nodes

BestFS — Problem Example (8-puzzle):

- 8-puzzle (Sliding Tiles)
  - $h_1(n)$: *Calculate number of misplaced tiles*
  - $h_2(n)$: *Calculate sum of **Manhattan distances** of all tiles from their goal positions*
  - Ignore moves that return you to the previous state

initial state:

2	8	3
1	6	4
7		5

goal state:

1	2	3
8		4
7	6	5

-

بالعربي: BestFS بيوسّع العقدة اللي عندها أقل h_1 (أقل عدد بلاطات غلط). بس مو دايماً يوصل لأقل تكلفة.

BestFS — Algorithm:

```
open = [start];
close = [ ];
while open ≠ [ ]
        remove leftmost state from open, call it X;
        if X  is goal return SUCCESS
        else
            generate children of X;
            put X on closed;
            discard children if on closed or open;
            put remaining children on left end of open;
            sort open by estimated remaining distance
return FAIL
```

بالعربي:

1. [] = open = [start], close
2. اسحب أول حالة من open → X
3. لو هدف → نجاح
4. وإلا → ولّد أبناءها، حط X في close، ارمي الأبناء اللي موجودين في close/open، حط الباقي في open، ورتب open حسب h
5. كرر

BestFS — Limitations:

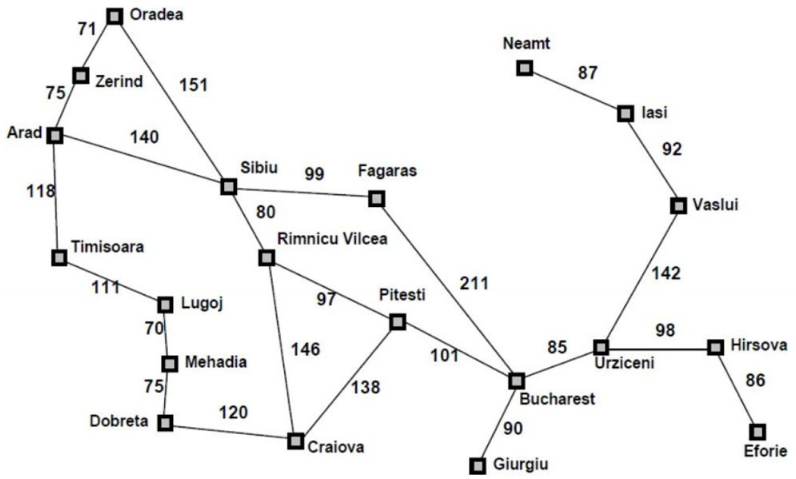
- It may not find the optimal solution
  - Only takes estimated cost from current state to goal (h)
  - Does not take actual cost from start to current state (g)
  - Already made bad choices can't be rectified

BestFS — Properties :

Property	Value	Justification
Complete	❌ No	It can get stuck in loops without repeated-state checking; complete in finite space with repeated-state checking
Time	O(b^m)	m = max depth; but a good heuristic can give dramatic improvement
Space	O(b^m)	Keeps all nodes in memory
Optimal	❌ No	Nodes expanded according to estimated remaining cost h(n), not actual path cost g(n)

بالعربي: BestFS مو مكتمل (ممكن يلف في حلقات) ومو أمثل (بيوسّع حسب h مو g). الوقت والمساحة O(b^m) — يحتفظ بكل العقد

BestFS — Student Exercise slide 27 if you want to see it :



straight-line distances to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

---

chapter 13

Part 3: A* Search | البحث A*

Introduction:

Basic idea: avoid expanding paths that are already found expensive.
New evaluation function: f(n) = g(n) + h(n)

- g(n) = cost so far to reach n (التكلفة الفعلية من البداية لـ n)
- h(n) = estimated cost from n to goal (التكلفة المتوقعة من n للهدف)
- f(n) = estimated total cost of path to goal via n (التكلفة الكلية المتوقعة)

A does not look only at "how far I have come" or only at "how close I seem to the goal." It considers both.*

Key Comparison:

Algorithm	Evaluation Function
A* Search	f(n) = g(n) + h(n)
Best-First Search	f(n) = h(n)
Uniform-Cost Search	f(n) = g(n)

A* — Problem Example (Romania):

Given a network of cities connected by roads, find a path from Arad to Bucharest using A* Search.
Heuristic: Straight-line distance (SLD / Euclidean) from city n to goal G.
Step-by-step :
Step 1: Initial node — Arad

- g(Arad) = 0
- h(Arad) = 366 (SLD, given)
- f(Arad) = 0 + 366 = 366

Step 2: Cost for Sibiu, Timisoara & Zerind

- f(Sibiu) = 140 + 253 = 393
- f(Timisoara) = 118 + 329 = 447
- f(Zerind) = 75 + 374 = 449

Step 3: Cost for Fagaras, Oradea & Rimnicu Vilcea (expanding Sibiu)

- f(Fagaras) = (140+99) + 176 = 415
- f(Oradea) = (140+151) + 380 = 671
- f(Rimnicu Vilcea) = (140+80) + 193 = 413

Continue: A* always expands the node with the smallest f — so it expands Rimnicu Vilcea (413) before Fagaras (415), etc. Eventually finds the optimal path.

A* — Algorithm:

```
Set PQ = {s}, CLOSED = { }, g(s) = 0, f(s) = h(s)// s is start
while (1){
  if PQ = { } return -1// fail
  n = PQ.dequeue(), Save n in CLOSED
  if n ∈ G return f(n) // success
    for each child, m, of n do {
      if m ∉ [PQ ∪ CLOSED]
         Set g(m) = g(n) + C(n,m)
         Set f(m) = g(m) + h(m)
         PQ.enqueue(m)
      if m ∈ [PQ ∪ CLOSED]
         Set g(m) = min (g(m), g(n) + C(n,m))
         Set f'(m) = g(m) + h(m)
         if f'(m) < f(m) & m ∈ CLOSED
           PQ.enqueue(m)
            f(m) = f'(m)  // improved path
    }
}
```

A* — Important Concepts :

1. Should we stop when we enqueue a goal?

No: only stop when we dequeue a goal.
Example :

Path	g	h	f
S	0	3	3
S→A	2	2	4
S→B	2	1	3
S→B→G	5	0	5
S→A→G	4	0	4

بالعربي: لما نضيف الهدف للـ queue، ممكن يكون في طريق أرخص لسه ما اكتشفناه. لازم نستنى الى ما نسحب الهدف من الـ queue (يعني تأكدنا إنه الأرخص).

2. Admissible Heuristic (حدسية مقبولة):

A heuristic h is admissible (optimistic) if:
h(n) ≤ C(n) — where C(n) is the true cost to the nearest goal
Examples: (showing h values less than actual costs)
Coming up with admissible heuristics is most of what's involved in using A in practice.*
بالعربي: الحدسية المقبولة هي اللي ما تزيد عن التكلفة الحقيقية (متفائلة). لو h أكبر من الحقيقي → A* ممكن يغلط.
Example of what goes wrong (from slides):
h = 6, h = 7, h = 0 — Actual bad goal cost < estimated good goal cost.
S→G = 5, h=0, f=5; S→A = 1, h=6, f=7.
We need estimates to be less than actual costs!
بالعربي: لو h بالغ في التقدير → A* ممكن يختار هدف سيء (تكلفته الحقيقية عالية) بدل هدف كويس. لازم h ≤ الحقيقي دايماً.

3. Consistent Heuristic (حدسية متسقة):

A heuristic h(n) is consistent if, for every node n and every successor n′ of n:
h(n) ≤ c(n, a, n′) + h(n′)
بالعربي: الحدسية المتسقة: التقدير من n للهدف ≥ تكلفة الخطوة لـ n′ + التقدير من n′ للهدف. يعني h مش بتزيد بشكل مفاجئ.
If h(n) is admissible and consistent, then A is complete and optimal.*
بالعربي: لو h مقبولة ومتسقة → A* مكتمل وأمثل.

A* — Conditions for Optimality:

First Condition: Admissible

- h(n) must be an admissible heuristic — never overestimates the cost to reach the goal (optimistic)
- h(n) ≤ C(n) — where C(n) is the true cost
- h_SLD(n) is admissible because the shortest path between two points is a straight line — h_SLD never overestimates actual distance
- If h(n) is admissible, f(n) never overestimates the cost to reach the goal because f(n) = g(n) + h(n) and g(n) is the actual cost

بالعربي: الشرط الأول: h مقبولة (ما تزيد عن الحقيقي). h_SLD مقبولة لأن الخط المستقيم هو أقصر مسافة.

Second Condition: Consistent

- h(n) must be a consistent heuristic
- h(n) ≤ c(n, a, n′) + h(n′) — for every node n and successor n′
- h_SLD(n) is consistent
- If h(n) is admissible and consistent, then A is complete and optimal*

بالعربي: الشرط الثاني: h متسقة (h(n) ≤ تكلفة الخطوة + h(n′)). لو الاتنين متحققين → A* مكتمل وأمثل.

Why consistency matters:

If h(n) is consistent, then the values of f(n) along any path are non-decreasing.
Proof: Suppose n′ is a successor of n; then g(n′) = g(n) + c(n, a, n′):
f(n′) = g(n′) + h(n′) = g(n) + c(n, a, n′) + h(n′) ≥ g(n) + h(n) = f(n)

- Whenever A* selects a node n for expansion, the optimal path to that node has been found.
- If not, there would be another frontier node n′ on the optimal path with lower f-cost — it would have been selected first.
- Hence, the sequence of nodes expanded by A* is in non-decreasing order of f(n).
- The first goal node selected for expansion must be an optimal solution (f is the true cost for goal nodes, h(Goal) = 0, and all later goal nodes will be at least as expensive).

بالعربي: لو h متسقة → f ما بتنقص على أي مسار. لما A* يختار عقدة، يكون لقى أحسن طريق لها. أول هدف بيتوسّع لازم يكون الحل الأمثل.

A* — Properties:

Property	Value	Justification
Complete	✅ YES	Unless there are infinitely many nodes with f ≤ f(Goal)
Time	Exponential	Depends on h(n)
Space	O(b^m)	Keeps all nodes in memory
Optimal	✅ YES	A* cannot expand f_{i+1} until f_i is finished

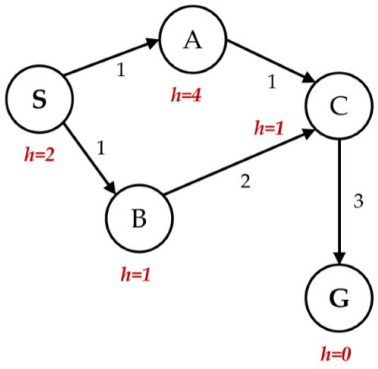
A* vs. Uniform-Cost :

Uniform-cost expands equally in all "directions"
A expands mainly toward the goal, but does hedge its bets to ensure optimality*

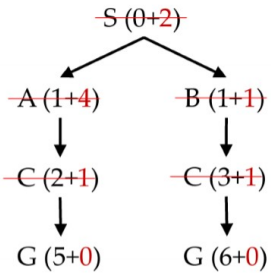
بالعربي: UCS يدور في كل الاتجاهات. A* يركز ناحية الهدف (بسبب h) بس يحافظ على الأمثلية (بسبب g).

A* — Graph Search Gone Wrong?:

State space graph



Search tree



Closed Set:S B C A

- Problem: A* graph search can go wrong if the heuristic is not consistent — it might find a suboptimal path because it closes a node before finding the optimal path to it. Solution: use a consistent heuristic (or reopen closed nodes if a better path is found).

  بالعربي: لو h مو متسقة، A* graph search ممكن يغلط — يقفل عقدة قبل ما يلاقي أحسن طريق ليها. الحل: h متسقة.

Summary

Algorithm	Evaluation Function	Based On	Complete?	Optimal?
Hill Climbing	h(n)	DFS	❌ No (local max)	❌ No
Beam Search	h(n)	BFS	❌ No (throws nodes)	❌ No
Best-First Search	f(n) = h(n)	DFS + BFS	❌ No (loops)	❌ No
A* Search	f(n) = g(n) + h(n)	—	✅ Yes	✅ Yes (admissible + consistent)
Uniform-Cost	f(n) = g(n)	BFS	✅ Yes	✅ Yes

Formulas:

A*: $f(n) = g(n) + h(n)$
BestFS: $f(n) = h(n)$
UCS: $f(n) = g(n)$
Admissible: $h(n) \leq C(n)$
Consistent: $h(n) \leq c(n, a, n') + h(n')$

Concept	Meaning
Admissible	h never overestimates true cost (optimistic)
Consistent	$h(n) \leq$ step cost + $h(n')$; f never decreases along a path
A* stops when	Goal is dequeued (not enqueued)
BestFS limitation	Uses h only — bad choices can't be rectified
Beam Search limitation	Throws away nodes — not complete

The Ideas:

1. Beam Search = BFS + keep only w best nodes — fast but not complete
2. BestFS = greedy — uses h only — fast but not optimal
3. A* = g + h — balances cost-so-far and estimated-remaining — complete and optimal if h is admissible + consistent
4. A* is optimal because it never expands a node with f higher than the optimal solution cost